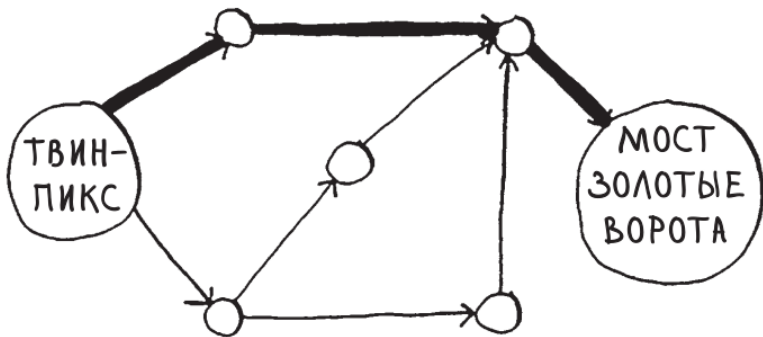
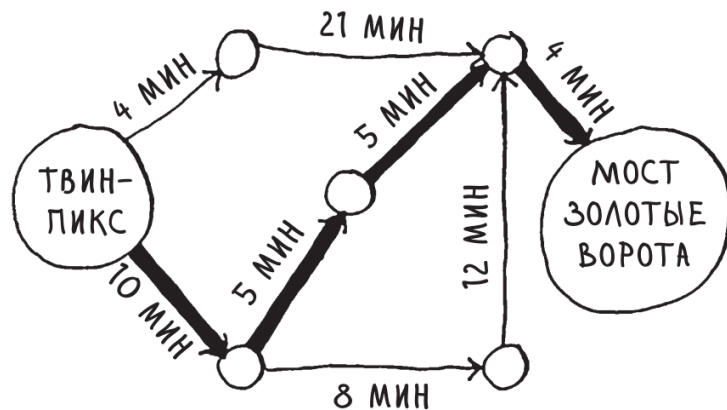


# Алгоритм Дейкстры

# Введение



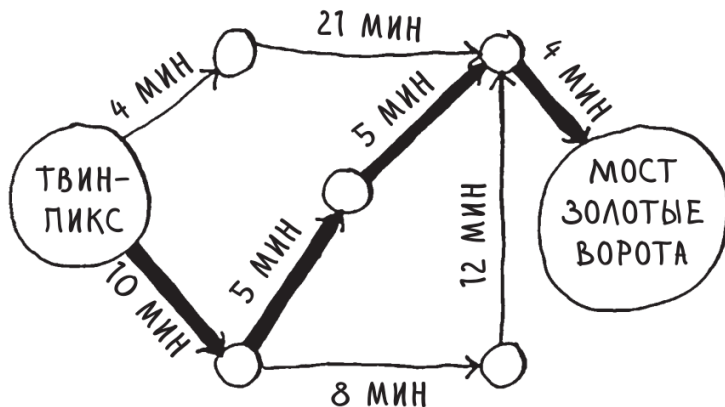
Поиск в ширину



Алгоритм Дейкстры

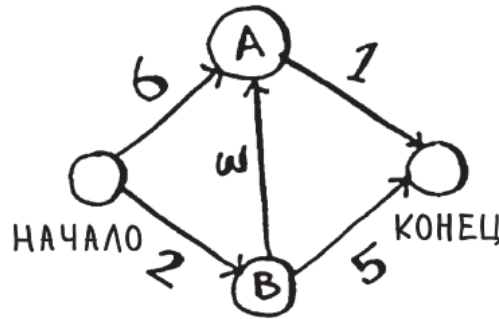
# Алгоритм Дейкстры

- Алгоритм Дейкстры решает задачу о кратчайшем пути из одной вершины во взвешенном ориентированном графе  $G = (V, E)$  в том случае, когда веса ребер неотрицательны.



# Работа с алгоритмом Дейкстры

- Посмотрим как этот алгоритм работает с графом

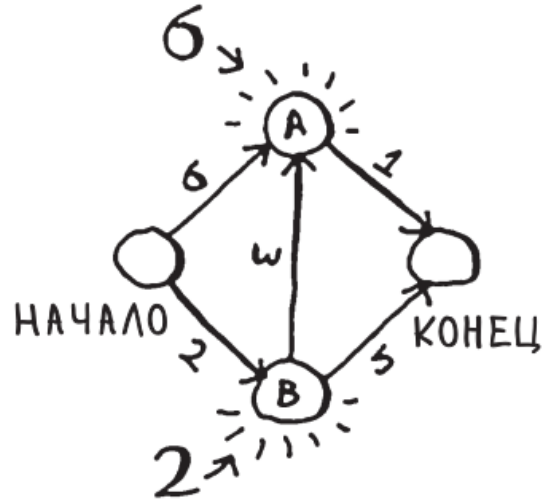


Каждому ребру назначается время перемещения в минутах. Алгоритм Дейкстры используется для поиска пути от начальной точки к конечной за кратчайшее возможное время.

# Работа с алгоритмом Дейкстры

- Алгоритм Дейкстры состоит из четырех шагов:
- 1. Найти узел с наименьшей стоимостью (то есть узел, до которого можно добраться за минимальное время).
- 2. Обновить стоимости соседей этого узла.
- 3. Повторять, пока это не будет сделано для всех узлов графа.
- 4. Вычислить итоговый путь.

# Шаг 1: найти узел с наименьшей СТОИМОСТЬЮ.



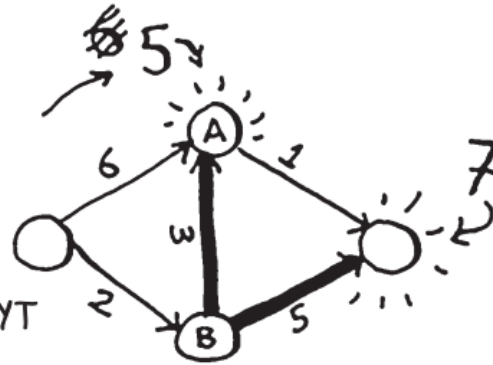
| УЗЕЛ  | ВРЕМЯ<br>ПЕРЕХОДА<br>К УЗЛУ |
|-------|-----------------------------|
| А     | 6                           |
| В     | 2                           |
| КОНЕЦ | $\infty$                    |

- До узла А вы будете добираться 6 минут, а до узла В — 2 минуты. Что касается остальных узлов, мы о них пока ничего не знаем.

# Шаг 2

- Вычислить, сколько времени потребуется для того, чтобы добраться до всех внешних соседей В при переходе по ребру из В.

| УЗЕЛ  | ВРЕМЯ          | ЧТОБЫ<br>ДОБРАТЬСЯ<br>ДО УЗЛА А,<br>ТЕПЕРЬ<br>ТРЕБУЕТСЯ<br>ВСЕГО 5 МИНУТ |
|-------|----------------|--------------------------------------------------------------------------|
| А     | <del>6</del> 5 |                                                                          |
| В     | 2              |                                                                          |
| КОНЕЦ | 7              |                                                                          |



Если вы нашли более короткий путь для соседа В, обновите его стоимость.

В данном случае мы нашли:

- более короткий путь к А (сокращение с 6 минут до 5);
- более короткий путь к конечному узлу (сокращение от бесконечности до 7 минут).

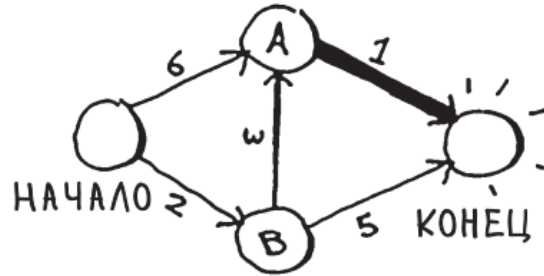
# Шаг 3: повторяем!

Снова шаг 1:

| УЗЕЛ ВРЕМЯ |   |
|------------|---|
| A          | 5 |
| B          | 2 |
| КОНЕЦ      | 7 |

С узлом B работа закончена, поэтому наименьшую оценку времени имеет узел A

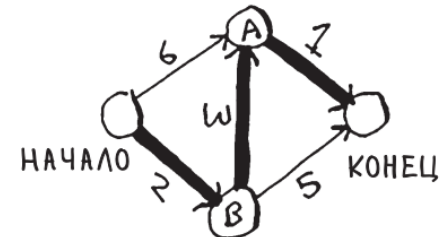
Снова шаг 2:



обновляем  
стоимости  
внешних  
соседей A.

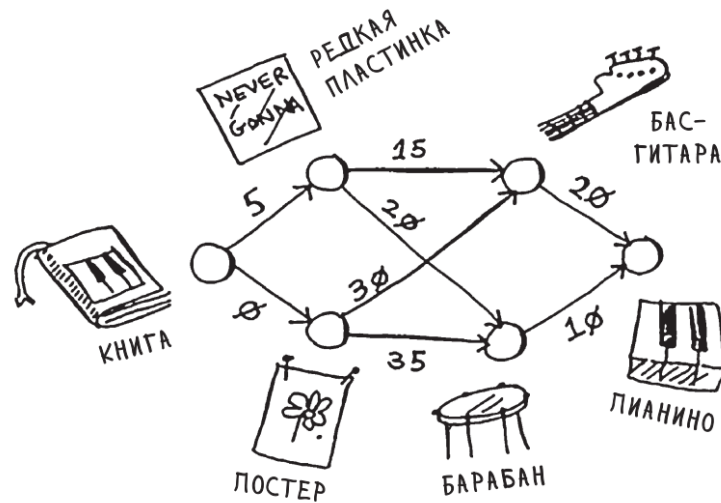
Алгоритм Дейкстры  
выполнен для каждого  
узла (выполнять его для  
конечного узла не нужно)!

| УЗЕЛ ВРЕМЯ |   |
|------------|---|
| A          | 5 |
| B          | 2 |
| КОНЕЦ      | 6 |

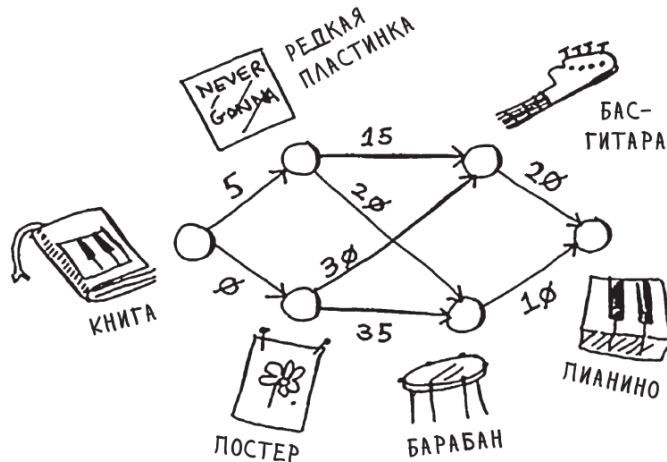


# Пример

- Необходимо совершить ряд сделок для обмена книги на пианино, минимизирую сумму доплат



# Пример. Начальные данные



УЗЕЛ СТОИМОСТЬ

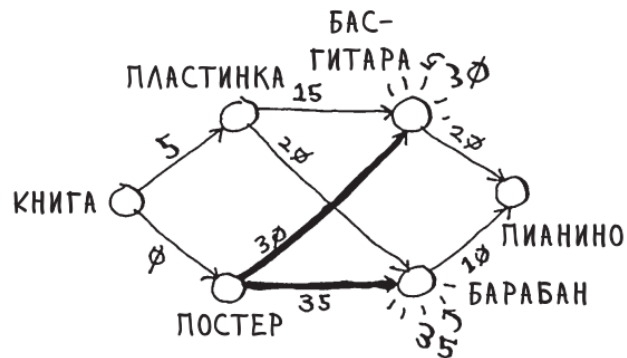
|           |   |
|-----------|---|
| ПЛАСТИНКА | 5 |
| ПОСТЕР    | 0 |
| ГИТАРА    | ∞ |
| БАРАБАН   | ∞ |
| ПИАНИНО   | ∞ |

МЫ ЕЩЕ  
НЕ ДОХОДИЛИ  
ДО ЭТИХ УЗЛОВ  
ОТ НАЧАЛЬНОГО

УЗЕЛ РОДИТЕЛЬ

|           |       |
|-----------|-------|
| ПЛАСТИНКА | КНИГА |
| ПОСТЕР    | КНИГА |
| ГИТАРА    | —     |
| БАРАБАН   | —     |
| ПИАНИНО   | —     |

# Пример. Постер.



| РОДИТЕЛЬ          | УЗЕЛ    | СТОИМОСТЬ     |
|-------------------|---------|---------------|
| КНИГА             |         | 5             |
| КНИГА             | ПОСТЕР  | 0             |
| <del>ПОСТЕР</del> |         | <del>30</del> |
| <del>ПОСТЕР</del> | БАРАБАН | <del>35</del> |
| —                 | ПИАНИНО | ∞             |

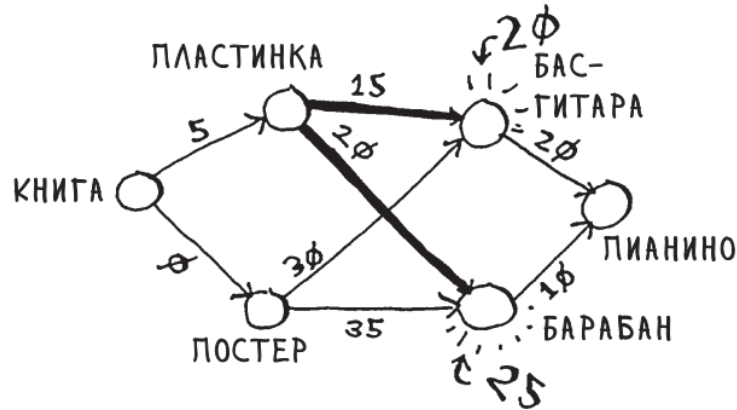
К ЭТИМ УЗЛАМ  
ПЕРЕХОДИМ ОТ  
УЗЛА «ПОСТЕР»

| РОДИТЕЛЬ | УЗЕЛ      | СТОИМОСТЬ |
|----------|-----------|-----------|
| КНИГА    | ПЛАСТИНКА | 5         |
| КНИГА    | ПОСТЕР    | 0         |
| ПОСТЕР   | ГИТАРА    | 30        |
| ПОСТЕР   | БАРАБАН   | 35        |
| —        | ПИАНИНО   | ∞         |

# Пример. Пластинка

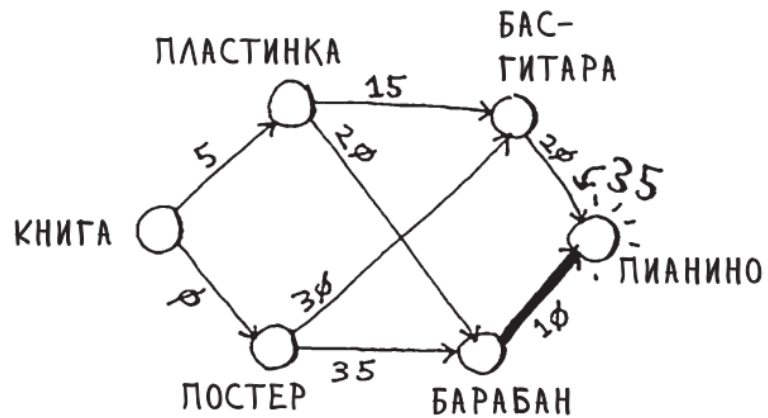
**Снова шаг 1:** пластинка — следующий по стоимости узел (\$5).

**Снова шаг 2:** обновляются значения всех его соседей.



| РОДИТЕЛЬ             | УЗЕЛ      | СТОИ-<br>МОСТЬ   |
|----------------------|-----------|------------------|
| КНИГА                | ПЛАСТИНКА | 5                |
| КНИГА                | ПОСТЕР    | 0                |
| <del>ПЛАСТИНКА</del> | ГИТАРА    | <del>20</del> 20 |
| <del>ПЛАСТИНКА</del> | БАРАБАН   | <del>25</del> 25 |
| —                    | ПИАНИНО   | $\infty$         |

# Пример. Барабан



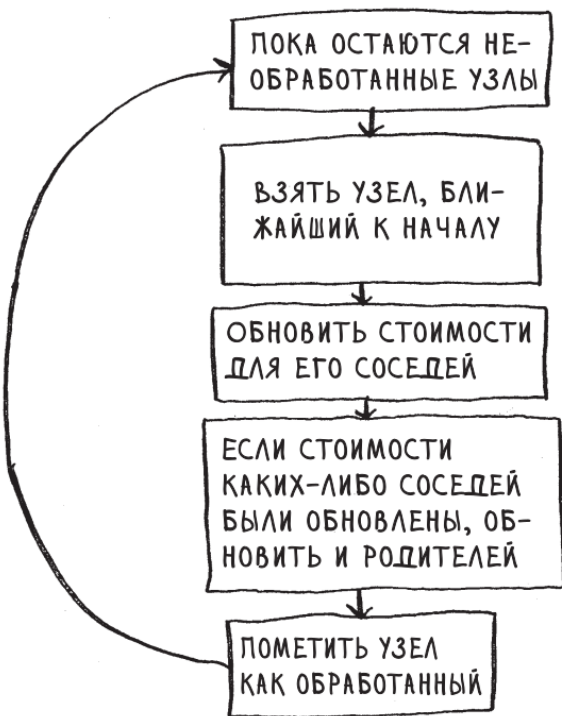
| РОДИТЕЛЬ  | УЗЕЛ      | СТОИ-<br>МОСТЬ |
|-----------|-----------|----------------|
| КНИГА     | ПЛАСТИНКА | 5              |
| КНИГА     | ПОСТЕР    | 0              |
| ПЛАСТИНКА | ГИТАРА    | 20             |
| ПЛАСТИНКА | БАРАБАН   | 25             |
| БАРАБАН   | ПИАНИНО   | 35             |

# Пример. Итог

- Серия обменов:

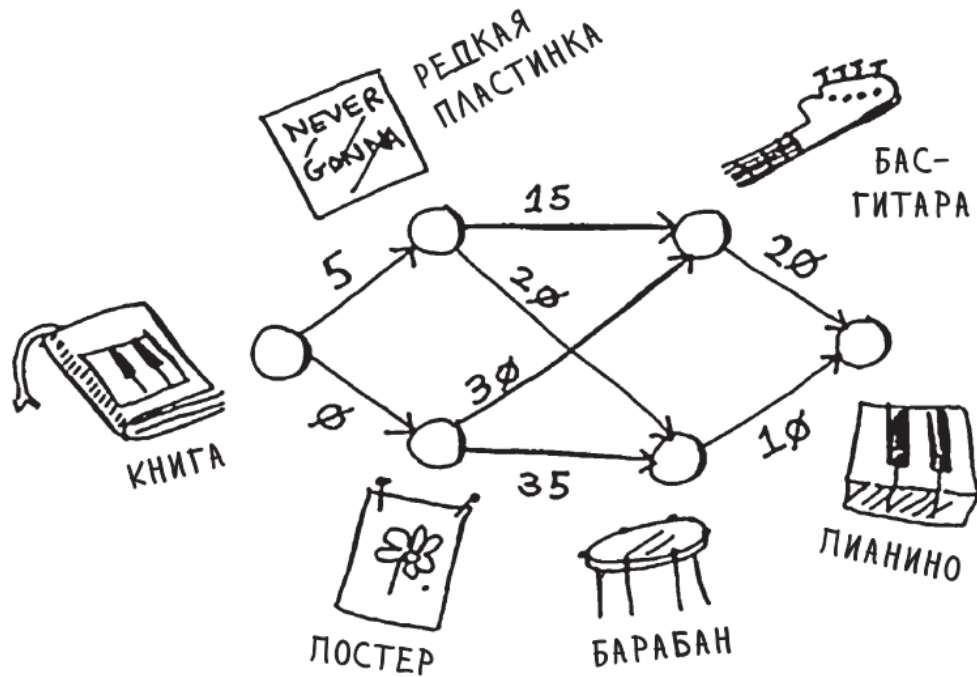


# Пример реализации



```
node = find_lowest_cost_node(costs)  ← Найти узел с наименьшей стоимостью среди необработанных
while node is not None:              ← Если обработаны все узлы, цикл while завершен
    cost = costs[node]
    neighbors = graph[node]
    for n in neighbors.keys():        ← Перебрать всех соседей (neighbors) текущего узла
        new_cost = cost + neighbors[n]  ← Если к соседу можно быстрее
        if costs[n] > new_cost:         ← добраться через текущий узел...
            costs[n] = new_cost         ← ...обновить стоимость для этого узла
            parents[n] = node           ← Этот узел становится новым родителем для соседа
    processed.append(node)            ← Узел помечается как обработанный
    node = find_lowest_cost_node(costs) ← Найти следующий узел для обработки и повторить цикл
```

# Реализовать пример



# Задача 1. Восстановление пути

- Условие:

Найдите кратчайший путь от вершины S до F и выведите его.

- Вход (граф):

```
S A 1
S B 4
A B 2
A C 5
B C 1
B D 3
C D 3
C F 2
D F 1
```

Выход:

```
Длина: 7
Путь: S → A → B → C → F
```

## Задача 2. Граф с несколькими путями одинаковой длины

- Условие:

В графе ниже два кратчайших пути от X до Y имеют одинаковую длину. Найдите их.

Вход:

```
X A 3
X B 2
A Y 1
B Y 2
```

Выход:

```
Длина: 4
Пути:
1. X → A → Y
2. X → B → Y
```

# Задача 3. Проверка на недостижимость

- Условие:

В графе есть вершины, недостижимые из A.  
Выведите -1 для них.

Вход:

|   |   |   |
|---|---|---|
| A | B | 5 |
| B | C | 3 |
| D | E | 2 |

Выход:

|    |    |
|----|----|
| A: | 0  |
| B: | 5  |
| C: | 8  |
| D: | -1 |
| E: | -1 |

# Задача 4. Реальный сценарий: маршрутизация в сети

- Условие:

Дана сеть маршрутизаторов. Найдите кратчайший путь по задержкам (в мс) от Router1 до Router5.

Вход:

```
Router1 Router2 10
Router1 Router3 20
Router2 Router4 50
Router3 Router4 20
Router4 Router5 10
Router2 Router5 100
```

Выход:

Длина: 60 мс

Путь: Router1 → Router3 → Router4 → Router5